Verifying Coordination in Parallel Coding Agents: NP-Bench and a Scheduling Planner

Anonymous Author(s)
Affiliation
Address
email

Abstract

A team of coding agents can look fine agent by agent and still fail as a team: each one passes its own tests while the merged result is broken, and single-agent evaluation never catches it. Teams increasingly run several LLM coding agents in parallel on the same codebase, and the agents get in each other's way: two rewrite the same function, one ships against a contract a teammate just changed, and integration fails after the work is already done. Most coordination tools treat this reactively (watch for a conflict, then warn), but under agent-speed concurrency the warning arrives after the wasted edit. We argue the problem is better posed as *scheduling*: take each work item's declared scope, partition the work into disjoint scopes, and order merges along the producer→consumer dependency graph, all up front. We build this planner into our coordination plane, a local-first system for coding agents, and evaluate it with NP-Bench, an environment-grounded, three-arm benchmark (no coordination; reactive detection; proactive planning) that verifies integration outcomes off a real git merge, both as a deterministic simulation and with live agents. Deterministically, the planner lifts clean-integration success from 1/9 to 9/9 scenarios and drives merge conflicts from 13 to 0; its advantage grows with the number of agents. With live agents on a breaking contract change, the planner rescues an outcome that both baselines miss on every seed: a clean-integration rate of 0 under no-coordination and under reactive detection rises to 1.0 with the planner on a frontier model, and to 0.6 on a small one, while agents respect their assigned scopes (none of 5 strayed). A separate memory result collapses the repeated-mistake rate from 1.00 to 0.00 on both a strong and a weak model: a decision recorded in one session stops an agent in a later session from repeating it, an information effect no amount of model strength substitutes for. We also report what did *not* help: at window-fitting scales, routing facts to agents does not rescue retrieval accuracy over long contexts; its value is cost and capacity, not attention. Methodologically, NP-Bench doubles as an environment-grounded *verifier* for multi-agent coding: each agent passes its own checks, yet the integrated system can still be broken, and a real merge-and-test reveals it. Across the two capability tiers and two vendors we tested, the benefit did not shrink as models got stronger, because it comes from how work is allocated rather than from model reasoning.

1 Introduction

Running one coding agent is now routine; running several at once is where teams are headed. Worktree multiplexers spin up an agent per branch, CI pipelines fan issues out to a pool of agents, and a single developer can supervise a handful of them at a time. Once that happens, the bottleneck stops being how well any one agent writes code and starts being whether the agents step on each

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

other. Two agents edit the same file and produce a merge conflict; one agent renames a field in an API while another codes against the old shape and integrates to something that merges cleanly but is broken; a decision made in the morning is re-litigated in the afternoon by an agent that never saw it. These are coordination failures, and they scale with the number of agents.

The prevailing response is reactive. A coordination layer watches for trouble (overlapping edits, a changed contract) and warns the affected agent once the change is observed. This is the same idea behind decades of developer "workspace awareness" tools [3], and it is genuinely useful for humans who work on the scale of hours. But agents work on the scale of seconds. By the time a teammate's edit has been sensed and a warning routed, the warned agent has often already written the conflicting code. The warning then buys a cleanup, not a prevention.

This paper takes a different stance: **coordination among parallel coding agents is a scheduling problem**. Before the agents start, take each work item's declared *scope*, partition the work so that concurrently-running agents have disjoint scopes, and order the merges along the producer→consumer dependency graph so that whoever changes a contract lands before whoever consumes it. Where a reactive detector can only tell an agent it has a problem after the fact, a schedule prevents the problem by construction.

There is an obvious objection: "disjoint scopes don't conflict" is a tautology. But that objection is the research question in disguise. A schedule helps only if (a) its input scopes are accurate and (b) the agents actually *respect* the scopes they are handed. In this paper the scopes are *declared* (given by each work item's stated files and consumed contracts), so we assume (a) and measure (b): an agent told to stay out of a file may edit it anyway, and whether it complies is exactly what the live experiments check. Inferring scopes automatically is an open problem we flag but do not solve. The contribution is the compliance measurement, not the tautology.

There is a verification problem hiding in that measurement. Whether a coordination layer actually helps is not something an agent can self-report, and it is invisible to single-agent evaluation: each agent, checked on its own branch, looks correct; the breakage surfaces only when the branches are integrated. Answering it takes an *environment-grounded* verifier that reads ground-truth signals (does the merged tree build, does it pass, did the consumer adapt to the new contract) off a real integration rather than trusting any agent's word. NP-Bench is that verifier, and its three-arm design turns "did coordination help" from an assertion into a measurement.

We make four claims and back each with data.

1. **A verification benchmark.** NP-Bench scores three arms (no coordination C0, reactive detection C1-DETECT, proactive planning C1-PLAN) both deterministically and with live agents from two model families, reading ground-truth integration signals off a real merge, the coordination axis that single-agent benchmarks like SWE-bench do not measure (Section 5). We present it as a focused study of *when* coordination helps, not a comprehensive benchmark.
2. **A formulation and a planner.** We cast coordination as disjoint-scope partitioning plus topological merge ordering, and implement a small, deterministic static planner (Section 4).
3. **Results with an honest capability analysis.** In the evaluated scenarios, the planner strictly dominates both baselines; a vague reactive warning is worth about as much as nothing to a live agent; and we separate the part of the effect that holds across the tested capability tiers from the part that varies with capability (Sections 6 and 8).
4. **What did not work.** We tried to show that routing facts to agents rescues retrieval accuracy over long contexts and could not; we report that negative result and the measurement bug we caught along the way (Section 7).

2 Related work

Multi-agent LLM frameworks. A first wave of systems orchestrates several LLM agents through roles and messages: AutoGen composes conversable agents under a group-chat manager [18]; MetaGPT runs an SOP-driven "software company" of agents [9]; ChatDev structures a chat-driven development organization [17]; CAMEL and AgentVerse study role-play and emergent multi-agent behavior [5, 13]. These divide labor by role and coordinate through dialogue. None offers a scheduling guarantee that concurrently-editing agents will avoid colliding on shared files or stale contracts. The planner here is complementary: it takes the work items such a framework produces and com-

Table 1: Where NP-Bench and the planner sit relative to adjacent work.

System / benchmark	Multi-agent	Coord. guarantee	Proactive	Parallel integration
AutoGen / MetaGPT / ChatDev / CAMEL	yes (roles)	no (emergent)	—	no
SWE-bench, HumanEval, LiveCodeBench	no (single agent)	n/a	n/a	no
Workspace-awareness / conflict warn	yes (humans)	no (warn only)	no (reactive)	partial
Build/merge DAG systems	n/a	yes (deps)	yes	no (not agents)
Agent memory (MemGPT, gen. agents)	partial	n/a	n/a	no
NP-Bench + planner (this work)	**yes**	**yes**	**yes**	**yes**

putes a disjoint-scope partition and merge order before execution. We target the substrate (who may touch what, and in what order), not the role or dialogue layer.

Coding-agent benchmarks. Evaluation of coding agents is dominated by single-agent, single-repository task success: SWE-bench resolves real GitHub issues [11], SWE-agent studies the agent-computer interface that drives such resolution [19], and execution benchmarks like HumanEval [4] and LiveCodeBench [10] score generated code. None measures whether N agents working in parallel integrate cleanly. NP-Bench adds exactly that axis (clean-merge success, wasted work, and scope compliance for *concurrent* agents) and is orthogonal to per-task resolution.

Conflict detection and developer coordination. Proactively predicting merge conflicts by watching parallel edits is a long line in software engineering [3]. This is the intellectual ancestor of the C1-DETECT arm: valuable, but reactive: the warning fires once a change is observed, which under agent-speed concurrency is frequently after the wasted edit. We show empirically that a proactive schedule dominates reactive detection, and that a vague reactive warning is close to no coordination for a live agent.

Scheduling and dependency-aware ordering. Framing coordination as scheduling connects to classic task-graph work: topological ordering of a dependency DAG [12] and the NP-hardness of the partitioning/coloring subproblem [7]. That software structure mirrors team structure is Conway's observation [6], and that adding people (or agents) to concurrent work raises coordination cost is Brooks's [2]. We adapt disjoint-partition-plus-topological-merge to the *declared code scopes* of LLM agents, where the novel empirical question studied here is whether agents will respect the resulting assignments. Inferring those scopes automatically remains future work.

Agent protocols and memory. Agent tooling is standardizing on protocols for model-to-tool and agent-to-agent communication—the Model Context Protocol [1] and Agent2Agent [8]. our coordination plane exposes its coordination surface over MCP but adds the missing piece those protocols leave open: what to route, to whom, and in what order. On memory, systems such as MemGPT [15] and generative agents [16] give an agent long-term recall within its own task. Our memory result is cross-session and cross-agent: a decision recorded by one session prevents an agent in a later session from repeating a mistake, and we isolate it as an information effect that holds equally on strong and weak models.

Table 1 places these lines side by side.

3 The coordination plane

The planner does not run in a vacuum; it needs to know what agents exist, what they are touching, and how services depend on one another. our coordination plane supplies that substrate. It is a local-first system for coding agents: a single per-machine daemon, a SQLite store, and a set of tools exposed over MCP (`register`, `sync`, `publish`, `decision`, `discover`, `chat`, `memory`, and `task`); the planner runs inside the daemon and delivers each agent's assignment through `register`/`sync`. It is agent-agnostic: Claude Code, Codex, and OpenCode all connect through the same surface.

The design choice that matters for coordination is that the daemon observes state rather than waiting to be told about it. A *passive sensing* loop polls each registered agent's git worktree and emits an event when the changed-file set actually changes, so coordination never depends on an agent

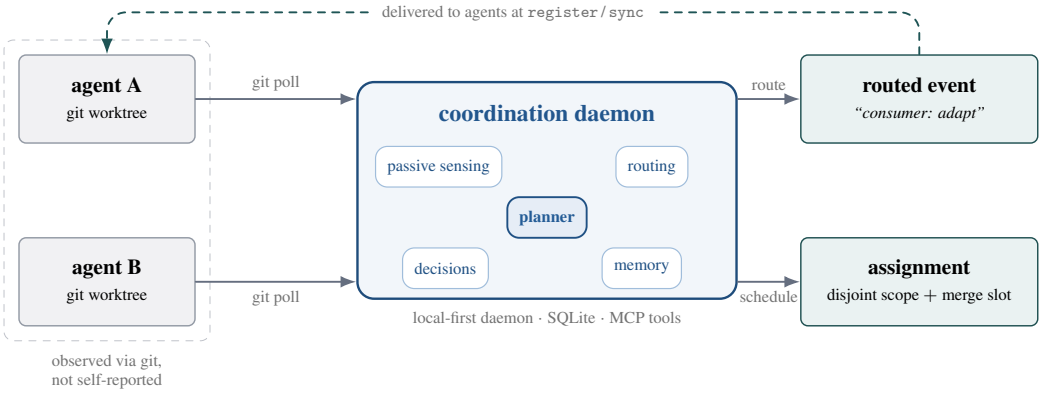

Figure 1: our coordination plane observes each agent's worktree by polling git (*passive sensing*), then routes only the relevant events and, with the planner, hands each agent a disjoint scope and a merge slot *before* the edits happen, rather than warning after a collision is sensed.

remembering to announce its work.[1] A routing engine then decides who needs to hear about a given change: the owner of an affected task, other agents in the same repository, and, through a service graph, agents in *other* repositories that consume a changed API contract. Durable project decisions live in a queryable ledger separate from chat history, and a cross-agent memory (keyword by default, optionally semantic) lets a fact learned in one session resurface in another. Figure 1 sketches the loop.

The contrast with a message bus is deliberate. A bus moves whatever agents choose to send; a state-aware plane understands branches, changed files, and service contracts, and can therefore route by relevance and, with the planner, act *before* the edits happen rather than after.

4 Coordination as scheduling

Model. Let W be a set of work items. Each item w has a *declared scope* $\sigma(w)$—the set of files (or symbols) it will write—and a set of contracts it *consumes*. Two items *conflict* when their scopes intersect, $\sigma(w_i) \cap \sigma(w_j) \neq \varnothing$. Item w_j *depends on* w_i (edge $w_i \to w_j$) when w_j consumes a contract that w_i produces, i.e. w_i's scope overlaps w_j's consumed set. A *plan* is a partition of W into batches whose members have pairwise-disjoint scopes, together with a merge order π that respects the dependency edges.

Claim (assumption-scoped). If the declared scopes are accurate and the dependency relation is a DAG, then a disjoint-scope partition plus a topological merge order yields *zero* structural conflicts by construction: concurrently-running items never touch the same file, and no consumer is merged before its producer. A reactive detector cannot make this guarantee because it acts only after a collision has been sensed. Scope accuracy is assumed through the hand-declared inputs; agent scope-compliance is the empirical unknown measured in Section 6.

Algorithm. Algorithm 1 is the static planner. It builds the dependency graph, marks items that must not take their naive edit (either because they contend for an owned file or because they consume a produced contract), computes a stable topological order by Kahn's algorithm [12] with deterministic cycle breaking, and greedily colors the scope-overlap graph into parallel batches. Exact minimum coloring is NP-hard [7]; the greedy pass is a documented approximation and is enough for the suite. Every tie breaks on input order, so a given input always yields the same plan, a requirement for reproducible experiments. The item's assignment (its scope, merge slot, and a human-readable reason when it was reassigned) is delivered to the agent at `register/sync` time.

The reassigned set R is what the suite ultimately probes. An item lands in R when the planner decides, up front and independent of runtime timing, that its naive scope would collide with a team-

[1] The system and the NP-Bench harness are released as open source (repository name, URL, and version withheld for anonymity).

Algorithm 1 BUILDPLAN (static coordination planner)

Require: work items W, each with scope $\sigma(w)$ and consumed set $c(w)$
Ensure: batches (disjoint-scope), merge order π, reassigned set R

build edges $w_i \to w_j$ for all $i \neq j$ with $\sigma(w_i) \cap c(w_j) \neq \varnothing$
$R \leftarrow \varnothing$; for each file, its first writer (input order) is owner
for all items w touching a file owned by another item **do**
    add w to R ▷ scope contention
end for
for all items w with ≥ 1 producer **do**
    add w to R ▷ must adapt to a consumed contract
end for
$\pi \leftarrow$ KAHN(edges) with input-order tie-break; break cycles by dropping the edge from the producer with fewest downstream consumers
greedily assign each item (in order π) to the lowest batch with no scope-overlapping item
return batches, π, R

mate or that it consumes a contract another item produces. That is exactly the information a reactive detector can only deliver after the agent has already edited.

5 NP-Bench

NP-Bench compares three arms. C0 is no coordination: agents act blind to one another. C1-DETECT is reactive coordination: an agent is warned when a teammate's sensed change touches a file it cares about, and (for live agents) the warning is deliberately vague ("a teammate may be editing X"), because that is what a realistic just-in-time warning looks like. C1-PLAN is the proactive planner: before starting, an agent receives either an allowed/forbidden scope or, for a contract consumer, the exact contract migration and an instruction to code against the new shape after the producer merges. The plan also supplies the merge order. Thus the live comparison evaluates the combined effect of timing, information specificity, scope assignment, and merge ordering rather than isolating any one component.

The suite runs at two tiers. **Tier A** is a deterministic simulation that drives the real product primitives—agent registration, worktree sensing, conflict detection, and the planner core—over scripted scenarios, and scores the result with a real git integration. An agent takes its coordinated edit exactly when it was warned in time (C1-DETECT) or reassigned by the planner (C1-PLAN); otherwise it takes its naive edit. Tier A is CI-gated and reproducible. **Tier B** replaces the simulated agents with live ones: real coding agents run headless in isolated git worktrees under each arm, their branches are integrated with a real merge, and the outcome is scored. Tier B is nondeterministic and costs money, so it is not a CI gate.

We report the following metrics. *CTSR* (clean-tree success rate) is the fraction of runs that both merge without conflict and pass a post-integration acceptance check. *Merge conflicts* and *wasted LOC* count git conflicts and the lines in conflicting edits that must be reworked. *Scope-leakage* is the fraction of reassigned C1-PLAN agents that edited a file they were told not to; 0 means full compliance. *Consumer-adaptation rate* is the fraction of fan-out consumers whose merged code uses the new contract field. *Repeated-mistake rate* is one minus adherence to a prior session's recorded decision. *Merge-order correctness* checks that every producer is ordered before its consumers.

Live runs span two model families: two Claude models of very different capability—a frontier model (Claude Opus 4.8, `claude-opus-4-8`) and a small one (Claude Haiku 4.5, `claude-haiku-4-5`)—and OpenAI's Codex (`gpt-5-codex`). The two Claude models separate what coordination does from what raw capability does; the second vendor's model checks that the effect is not a Claude-specific quirk. Live sample sizes are modest ($K = 3$–15 seeds per cell, the pivotal contract cell at $K{=}15$); we report Wilson 95% intervals and treat the smaller cells as pilots (Section 9).

Table 2: NP-Bench headline. C1-PLAN dominates both baselines; C1-DETECT≈C0 on every live metric. Tier A is deterministic; Tier B is live agents.

#	Experiment (metric)	C0	C1-DETECT	C1-PLAN	Tier
1	Deterministic 3-arm, 9 scenarios (CTSR)	1/9	4/9	**9/9**	A
1	merge conflicts (total)	13	12	**0**	A
1	wasted LOC (total)	54	49	**0**	A
2	Effect-by-N contention, N=8 (wasted LOC)	28	28	**0**	A
3	Live contract migration, Opus 4.8 (CTSR)	0.00	0.00	**1.00**	B
3	Live contract migration, Haiku 4.5 (CTSR)	0.00	0.00	**0.60**	B
3	Live contract migration, GPT-5-Codex (CTSR)	0.00	0.00	**1.00**	B
4	Live fan-out, $N\in\{2,4,8\}$ (adaptation)	0.00	0.00	**1.00**	B
5	Same-file assignment, Opus 4.8 (scope-leakage)	—	—	**0.00**	B
6	Memory continuity, both models (repeated-mistake)	1.00	—	**0.00**	B

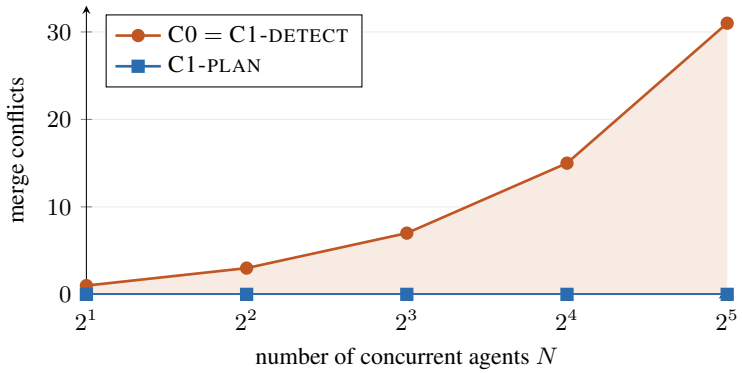

Figure 2: Deterministic effect-by-N (contention on one file). C1-DETECT tracks C0 because the reactive warning is always too late under concurrency; C1-PLAN holds at zero conflicts at every scale. Wasted LOC follows the same shape at $\approx 4(n-1)$.

6 Results

Table 2 consolidates the headline numbers; the rest of this section reads them off one experiment at a time.

6.1 Deterministic three-arm and effect-by-N

Across nine scripted scenarios (shared-file, contract, cross-repo fan-out, an independent control, and concurrent/high-N variants), CTSR rises from 1/9 under C0 to 4/9 under C1-DETECT to 9/9 under C1-PLAN; total merge conflicts fall 13→12→0 and wasted LOC 54→49→0. The gap between the two coordination arms is the point. On *sequential* dependency scenarios the reactive detector and the planner tie: both fix the problem, because the warning has time to land. On *concurrent* scenarios the detector collapses to the C0 baseline: the warning arrives after the edit. The planner does not, because it never let the collision be scheduled in the first place. On the control scenario (unrelated tasks) all three arms succeed and the plane raises no warnings, so the coordination layer does no harm when there is nothing to coordinate. Cross-repo routing is exact: on the fan-out scenario the two real consumers are warned and the unrelated service is not (100% routing hit, 0% false positives).

The effect grows with the number of agents. Figure 2 sweeps N agents concurrently rewriting one hot file. Conflicts scale as $n-1$ and wasted LOC as $\approx 4(n-1)$ for both C0 and C1-DETECT; the planner holds at zero across $N \in \{2, 4, 8, 16, 32\}$. The reactive detector tracks no-coordination exactly here—under concurrency its warning is always too late—so the advantage of planning is monotone in N.

Table 3: Live contract-migration CTSR across two model families, with Wilson 95% CIs on the planner arm. The result holds across the tested capability tiers and two vendors (all three models fail at 0.00 without the signal); acting on the plan varies with capability (the two frontier models 1.00, the small model 0.60). On the frontier model the planner and baselines separate with non-overlapping intervals (15/15 vs. 0/15; Fisher's exact $p < 10^{-4}$).

model	K	C0	C1-DETECT	C1-PLAN	C1-PLAN 95% CI
Claude Opus 4.8 (`claude-opus-4-8`)	15	0.00	0.00	**1.00**	$[0.80, 1.00]$
Claude Haiku 4.5 (`claude-haiku-4-5`)	5	0.00	0.00	**0.60**	$[0.23, 0.88]$
OpenAI GPT-5-Codex (`gpt-5-codex`)	5	0.00	0.00	**1.00**	$[0.57, 1.00]$

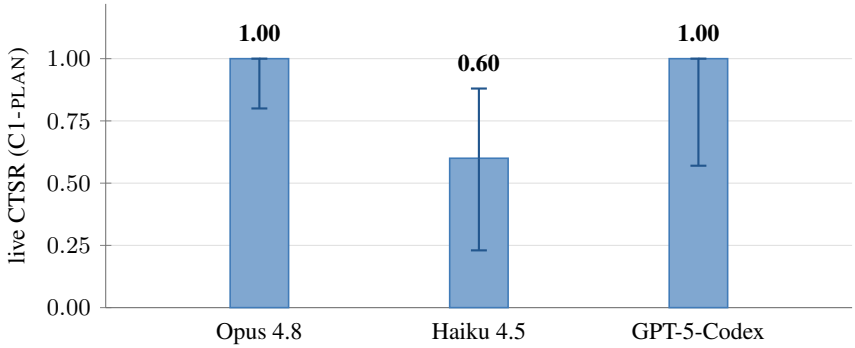

Figure 3: Live contract-migration CTSR of the planner (C1-PLAN) with Wilson 95% CIs, by model (Claude Opus 4.8, Claude Haiku 4.5, OpenAI GPT-5-Codex); both baselines (C0, C1-DETECT) sit at the floor, 0.00, for every model. Opus 4.8 is $K{=}15$, the others $K{=}5$.

6.2 Live agents: the planner lifts clean-integration outcomes

Deterministic results can be dismissed as a simulation we built, so the live runs are the main evidence. In the contract-migration scenario, a producer renames an invoice money field (`amount` in dollars $\rightarrow$ `amountCents` in integer cents); the consumer, working from the old contract in its own worktree, implements a total. The two files merge cleanly, but the consumer is semantically broken unless it adapts. Table 3 gives CTSR for three models across two vendors (the pivotal frontier cell at $K{=}15$, the others at $K{=}5$) with Wilson 95% confidence intervals on the planner arm.

The planner lands the breaking change where both baselines fail on every seed (the frontier models) or most seeds (Haiku). C1-DETECT does no better than C0, sitting at 0.00 on every model: the raw replies show the consumer knew "something may change" and still coded to the stale field. A heads-up without the specifics and a merge order buys nothing; only the explicit up-front assignment carries the contract shape the agent needs. We read this as the *combined* planner intervention (up-front timing together with the exact scope and contract) beating a vague, late warning. The live setting does not isolate timing from information specificity, and the detailed-but-late ablation that would (Section 9) is future work. The failure of C0 is not a capability failure: the information the consumer needs is simply not in its worktree, so no amount of model strength recovers it, and the same pattern holds across a second vendor's model, so it is not a Claude-specific artifact.

The fan-out version scales that scenario: one producer migrates the contract and N consumers each render in their own file. Adaptation rate is 0.00 under both baselines and 1.00 under the planner at every $N \in \{2, 4, 8\}$ (frontier, $K{=}3$). The rate saturates, so the absolute number of broken consumers under C0 grows linearly with N while the planner holds all of them.

Finally, the make-or-break question for the whole approach: do agents respect the scopes they are assigned? On the shared-file scenario ($K{=}5$, frontier), scope-leakage is 0.00: every planner agent put its new work in a separate file and left the owned file untouched, several of them noting that they were doing so "as coordinated." The disjoint-scope partition is not a tautology defeated by non-compliance; real agents honor it.

6.3 Memory continuity: a capability-independent effect

The results above concern work allocation. A separate experiment concerns knowledge. No model, however strong, can recall a project-specific fact it was never told; if one session discovers a gotcha and records it, an agent in a later session should stop repeating the mistake—but only if memory carries the discovery across sessions. We built a sandbox repository with two symmetric authentication backends that both export the same function, where which one is sanctioned appears nowhere in the code; it is a team decision that lives only in memory. Session 1 records the decision; session 2 (a fresh session) is asked which backend to import. The repeated-mistake rate falls from 1.00 to 0.00 on both the frontier model and Haiku ($K{=}5$). Because the correct choice is not recoverable from the repository, the gap is an information gap, not a reasoning gap, and it closes identically regardless of model strength. The raw replies confirm it: without memory the agents call the two backends "a genuine tie" and guess; with memory they cite the recalled policy and import the sanctioned module.

7 What did not work

We initially hypothesized a second result: that routing a needed fact to an agent would rescue its *accuracy* over a long, distracting context, an "attention rescue" to complement the lost-in-the-middle literature [14]. It did not hold. We report it anyway, because it changes what one should claim for context routing.

At the modest scales we first tested, a frontier model recalled a fact buried at any position with 100% adherence, with no U-shaped dip. We then built a harder, size-swept test: a single agent must retrieve the current payments contract, buried at $\sim$50% depth among four hard-negative money shapes, in a codebase inflated to a target token budget, against routing that delivered the one relevant fact ($\sim$155 tokens). Accuracy did not degrade with dilution: both the frontier model and the small one picked the current contract over the decoys at 10k, 50k, and 100k tokens (and the frontier model up to $\sim$280k). A still-harder variant requiring a three-fact join with named traps gave the same answer: no accuracy gap opened, even for the small model.

What is real, and large, is cost and capacity. Routing delivers the needed fact in $\sim$155 tokens versus 10k–280k dumped into a single agent's prompt—68$\times$ to 1296$\times$ less context per task—and the single-agent prompt that carries an entire microservices world ($\sim$280k tokens) overflows the small model's 200k window outright (5/5 failures), while the per-agent routed prompt always fits. So per-repo scoping plus routing is a cost-and-feasibility win, not an accuracy rescue; we state it that way rather than the stronger claim we could not support.

One methods note belongs here, because it is the reason we trust the numbers above. An earlier run of the decision-adherence test reported a 70% versus 100% effect on a frontier model. It was a scorer artifact: a regular expression that negated on a substring false-flagged correct replies. Structured output scoring plus raw-reply logging erased the effect, and we retract that number. The lesson—score structured fields and log raw replies rather than pattern-match prose—is now baked into the harness, and it is why the positive results in Section 6 are scored on parsed artifacts (the actual import line, the merged field) rather than on prose mentions.

8 Discussion

The live contract result splits cleanly in two. The coordination *signal* helps across the tested capability tiers: without it, a frontier model, a small model, and a second vendor's model all fail at 0.00, because the fact is absent from their context, not beyond their reasoning. *Acting* on the signal is partly capability-dependent: given the same plan, the frontier models adapt perfectly (1.00) and the small model most of the time (0.60), with the misses tracing to weaker execution (a stray edit, an incomplete migration) rather than to missing information. That split matters for anyone worried that better models will make coordination unnecessary. On the evidence here the information half is not closed by scale; the execution half already benefits from it.

It also explains when reactive detection is enough and when it is not. In the deterministic timing model, a warning lands in time on sequential dependencies and the detector matches the planner. Under scripted concurrent contention the warning arrives after the edit, so the detector tracks no-coordination and the gap to the planner grows with N. In the live study, where timing and informa-

tion specificity change together, the evidence supports the combined proactive intervention rather than scheduling alone.

Verification is easiest where ground truth is unambiguous—formal proofs, competitive programming, passing test suites. Multi-agent code integration is one more such place: the merged tree either builds and passes its acceptance check or it does not. NP-Bench exploits that to *verify* coordination end to end—it runs real agents in a real git integration and reads ground-truth signals (clean-merge success, scope compliance, contract adaptation, cross-session decision adherence) directly off the merged tree, rather than trusting each agent's self-report. Coordination failures are invisible to single-agent verification: each agent, judged alone, did fine; the system still broke on integration. The three-arm design (nothing / reactive / proactive) lets us measure whether the coordination layer helped instead of asserting it, and the negative arm guards against overclaiming.

9 Threats to validity

Tier A is a deterministic simulation of agent *reaction*; it drives the real sensing, detection, routing, and planner, but the agent edits themselves are scripted, so it validates mechanism and routing precision, not agent behavior. That is what Tier B is for. The live cells span two model families (Claude frontier/Haiku and OpenAI Codex), which guards against a single-vendor artifact, and the pivotal contract cell is run at $K{=}15$, where the planner's Wilson interval $[0.80, 1.00]$ separates cleanly from the baselines' $[0.00, 0.20]$ (Fisher's exact $p < 10^{-4}$). The remaining live cells are $K{=}5$ pilots with wider intervals, and the scenarios are hand-constructed; larger K across every cell and a broader scenario set are the obvious next runs.

The most important limitation is scope inference. In these experiments the declared scopes are *hand-seeded* in the harness (each item's stated files and consumed contracts), not inferred from natural-language tasks. The planner's guarantee is only as good as those scopes, and inferring them automatically (from a symbol graph or the service graph) is unbuilt work, not a solved component; we claim the scheduling result, not automatic scope inference. Scope-leakage is also measured on the simple shared-file case; leakage could be higher on subtler scopes. Finally, the C1-DETECT warning is deliberately vague. We argue this is realistic rather than a straw man—a just-in-time "someone may be touching X" is what reactive systems actually emit—but a specificity-versus-timing ablation (a detailed but late warning) would sharpen the comparison and is future work.

10 Conclusion

Parallel coding agents collide at integration, and that collision is a scheduling problem. Given declared scopes, scheduling around them (disjoint scopes for concurrent work, producers merged before their consumers) turns conflict cleanup into conflict prevention. On NP-Bench the planner lifts deterministic clean-integration from 1/9 to 9/9 and, with live agents from two vendors, rescues a breaking contract change that no-coordination and reactive detection both miss (0.00/0.00→1.00 on the frontier models, 0.60 on a small one), while agents respect their assigned scopes. A cross-session memory closes a repeated-mistake gap identically on strong and weak models. Routing facts to agents, which we expected to rescue long-context accuracy, does not at window-fitting scales; its value is cost and capacity. The benefit held across the models we tested because it comes from how work is allocated, not from model reasoning. What remains open is inferring scopes automatically rather than hand-seeding them, re-planning as work changes, and moving from advisory scopes to enforced ones.

Reproducibility. The system and the NP-Bench harness are released as open source (repository withheld for anonymity). Tier A is deterministic and CI-gated; Tier B is a set of live-agent runs that are nondeterministic and reported with Wilson 95% confidence intervals.

References

[1] Anthropic. Introducing the model context protocol. `https://www.anthropic.com/news/model-context-protocol`, 2024. Accessed 2026-08-20.

[2] Frederick P. Brooks. *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley, 1975.

[3] Yuriy Brun, Reid Holmes, Michael D. Ernst, and David Notkin. Proactive detection of collaboration conflicts. In *ACM SIGSOFT FSE*, pages 168–178, 2011.

[4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[5] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. *ICLR*, 2024.

[6] Melvin E. Conway. How do committees invent? *Datamation*, 14(4):28–31, 1968.

[7] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[8] Google. Announcing the Agent2Agent protocol (A2A). `https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/`, 2025. Accessed 2026-08-20.

[9] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. *ICLR*, 2024.

[10] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *ICLR*, 2025.

[11] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *ICLR*, 2024.

[12] Arthur B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5(11): 558–562, 1962.

[13] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for "mind" exploration of large language model society. *NeurIPS*, 2023.

[14] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the ACL (TACL)*, 12:157–173, 2024.

[15] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[16] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *ACM UIST*, 2023.

[17] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. *ACL*, 2024.

[18] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed H. Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[19] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *NeurIPS*, 2024.